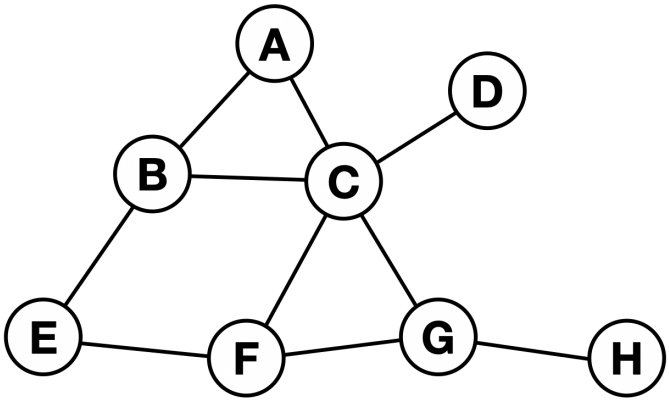
Multimodal Deep Learning

Solutions to Exercise Sheet 5

Date: 17th of June, 2025

Exercise 1: Graph representations



Given is above's graph with the vertices $\mathcal{V} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}, \mathbf{E}, \mathbf{F}, \mathbf{G}, \mathbf{H}\}$.

(a) Write down its representation as list of edges $\mathcal{E}$.

(b) Write down its representation as the adjacency matrix $\mathcal{A}$.

Solution

(a) $\mathcal{E} = \{(\mathbf{A}, \mathbf{B}), (\mathbf{A}, \mathbf{C}), (\mathbf{B}, \mathbf{C}), (\mathbf{B}, \mathbf{E}), (\mathbf{C}, \mathbf{D}), (\mathbf{C}, \mathbf{F}), (\mathbf{C}, \mathbf{G}), (\mathbf{E}, \mathbf{F}), (\mathbf{F}, \mathbf{G}), (\mathbf{G}, \mathbf{H})\}$

(b) The adjacency matrix, in which the rows and columns are organized by lexical order of the node names, is given by: $\mathcal{A} = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$.

Exercise 2: Graph scaling

Assume a social graph of 1,000,000 members (nodes) and that each node is connected to 200 randomly selected other nodes.

(a) How large is the adjacency matrix of this graph?

(b) How many entries of the adjacency matrix are non-zero?

(c) Given a random node, estimate how many hops are required until 50% of the network can be reached?

(d) Why may this estimation not hold in real-world social graphs?

(e) How would above's scaling behavior affect the application of graph convolutional neural networks?

Solution

(a) The adjacency matrix is $10^6 \cdot 10^6 = 10^{12}$ entries large.

(b) Assuming the diagonal is filled with zeros, $200 \cdot 10^6 = 2 \cdot 10^8$ non-zero entries, less than 0.02% of all entries.

(c) A naive approach would estimate the fraction of connected nodes after k hops as: $S_k = \frac{200^k}{1{,}000{,}000}$. However, this neglects that nodes may connect to a previously visited node as the number of hops increases. This behavior can be approximated using a mean-field "epidemic-like" approximation: $S_{k+1} = 1 - e^{-200 * S_k}$. This yields $S_0 = 10^{-6}$, $S_1 \approx 0.0020\%$, $S_2 \approx 3.9206\%$, $S_3 \approx 99.9606\%$. An alternative approach is to simulate random graphs with the corresponding properties and running a breadth first search (see included code snippet `bfs.py`). Both solutions yield that 3 hops already cover substantially more than 50% of nodes.

(d) The number of neighbors will vary a lot in social graphs dependent on the social circle of individual users and their engagement with the platform. Even if we presume that every node is connected to exactly 200 edges, there will be a substantial clustering among social circles.

(e) As the number of graph convolutional layers increases, the learned node representations are shaped by essentially the whole graph. As such the network is unable to distinguish between different nodes and produces similar node representations for all nodes.

Exercise 3: Homogeneous vs. heterogeneous graph convolutional networks

In the lecture we introduced the message passing update rule for a given node of graph convolutional neural networks as:

$$v_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}_i} \frac{1}{c_i} W^{(l)} v_j^{(l)} + W_0^{(l)} v_i^{(l)}\right) \tag{1}$$

However, for homogeneous graphs it is often written as:

$$v_i^{(l+1)} = \sigma\left(W^{(l)} \sum_{j \in \mathcal{N}_i} \frac{1}{c_i} v_j^{(l)} + W_0^{(l)} v_i^{(l)}\right) \tag{2}$$

Finally, recall that the node update rule for heterogeneous graph convolutional neural networks is:

$$v_i^{(l+1)} = \sigma\left(\sum_{r \in R} \sum_{j \in \mathcal{N}_i^r} \frac{1}{c_{i,r}} W_r^{(l)} v_j^{(l)} + W_0^{(l)} v_i^{(l)}\right) \tag{3}$$

(a) What do the variables $v_i^{(l)}, v_j^{(l)}, \mathcal{N}_i, W^{(l)}, W_0^{(l)}, \sigma$ and $\sum_{j \in \mathcal{N}_i}$ correspond to?

(b) How do the networks described by Equations 1 and 2 differ? Do these differences have a large impact on their performance?

(c) What do the variables $W_r^{(l)}$ and $\mathcal{N}_i^r$ correspond to?

(d) Why is Equation 1 used as basis when extending a homogeneous to a heterogeneous graph neural network instead of Equation 2?

Solution

(a) $v_i^{(l)}$ the node that will be updated at step (or network layer) (l) .
$v_j^{(l)}$ are the neighboring nodes connected to $v_i^{(l)}$.
The set of these nodes is the neighborhood $\mathcal{N}_i$.
$W^{(l)}$ denotes the weights the neural network that processes the neighboring nodes' signals.
$W_0^{(l)}$ is the weight matrix of the neural network that processes the node's current state.
σ is a non-linearity.
$\sum_{j \in \mathcal{N}_i}$ aggregates either the processed (Equation 1) or processed and aggregated (Equation 2) signals.

(b) Equations 1 and 2 differ in the order of the neural network and aggregation function. Dependent on the size of the weight matrix $W^{(l)}$, aggregating before applying the neural network may be computationally more efficient. As long as W is a simple matrix without any non-linearities, their order does not matter.

(c) $W_r^{(l)}$ denotes the weights of a neural network that is designed to process the signal of neighboring nodes with a certain modality r.
$\mathcal{N}_i^r$ is the set of neighboring nodes with the specific modality r.

(d) Aggregating signals from different node types before processing would require different weight matrices dependent on the exact composition of the node types. With increasing number of neighbors and/ or modalities, this number combinatorially explodes.

Exercise 4: GNNs, CNNs and transformers

The lecture discussed that CNNs and transformers are both special cases of graph convolutional neural networks.

(a) What is the adjacency matrix for a given pixel and a 3×3 convolutional kernel?

(b) Does a CNN rather resemble a homogeneous or a heterogeneous graph convolutional neural network (cf. Equations 2 and 3 above)?

(c) How would the connectivity graph of a vanilla transformer look like?

(d) What network components are missing to extend a graph convolutional neural network as described in Equation 2 to a transformer architecture?

(a) If we place the central pixel in the fifth row/ column, $\mathcal{A} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$.

(b) Depending on the position of the eight neighboring pixels, different kernel entries are used to process their respective signals. This is similar to a heterogeneous graph convolutional neural network processing different modalities.

(c) It resembles a graph in which every node is connected to all other nodes.

(d) The most prominent missing component is an attention mechanism. Adding this to a basic graph convolutional neural network yields graph attention networks (cf. Veličković et al. "Graph Attention Networks." International Conference on Learning Representations. 2018. https://arxiv.org/abs/1710.10903). Additionally, transformers include normalization layers and an added feed-forward multi-layer perceptron after each layer.